■ ALASSIO DATA CATALOG

Deployment Guide · Ubuntu

Installation on your own Ubuntu server · Version 1.79.6

This guide takes a freshly installed Ubuntu server to a running installation of the Alassio Data Catalog: database, Node.js application server with the MCP server for AI agents behind nginx, the optional Python inspector for reading source systems, HTTPS with Let's Encrypt and the first administrator account. Allow 45 to 60 minutes.

Every command is on its own line and can be run in order. Replace placeholders in angle brackets (such as `<password>`) with your values. Files are edited with `vim`.

Target system	Ubuntu 22.04 LTS or later, x86_64 or ARM64 (e.g. Orange Pi), 2 CPU cores, 4 GB RAM, 10 GB free disk
Software	Node.js 24 LTS (at least 22.12), MariaDB 10.6+ or PostgreSQL 14+, nginx, Python 3.10+ (inspector only)
Package	`datahub-catalog-deploy.zip` (v1.79.6) or a clone of the Git repository
Result	https://<your-host>/ with sign-in, catalog, marketplace, MCP endpoint /mcp and health check /healthz

Overview: what runs where

Component	Purpose	Location / port
nginx	serves the UI, forwards API, MCP, OAuth and health check to the Node server, TLS	/var/www/datahub-catalog · :80 / :443
Application server (Node.js)	sign-in, permissions, catalog storage, workflows, imports/exports, Unity Catalog sync, MCP server, scheduler	/opt/datahub-catalog · 127.0.0.1:3000 · systemd: datahub-catalog-proxy
MariaDB (or PostgreSQL)	users, sessions, catalog (tables cat_*), audit trail, jobs, settings	127.0.0.1:3306 (5432)
Source inspector (Python, optional)	reads metadata from DuckDB, data lake files, SQL databases (SQLAlchemy) and JDBC sources	/opt/datahub-catalog/inspector · 127.0.0.1:3001 · systemd: datahub-catalog-inspector
Data directory	upload temp files, local backups	/var/lib/datahub-catalog

Note: Only nginx is reachable from outside. The application server, inspector and database bind to 127.0.0.1 and must not be opened in the firewall.

Before you start

- Root or sudo access via SSH.
- The release package `datahub-catalog-deploy.zip` in the server's home directory (e.g. via `scp`).
- A DNS name for HTTPS, e.g. `catalog.example.com`, pointing to the server.
- Optional: SMTP credentials for invitations and notifications, a Databricks workspace for the Unity Catalog import, credentials of the source systems for the inspector.

Step 1 · Base system and Node.js 24

```
sudo apt update
sudo apt upgrade -y
sudo apt install -y nginx unzip rsync curl ca-certificates gnupg python3-venv
curl -fsSL https://deb.nodesource.com/setup_24.x -o nodesource_setup.sh
sudo bash nodesource_setup.sh
sudo apt install -y nodejs
node -v
```

Expected: `v24.x`. The application requires at least Node.js 22.12; older versions do not start.

Step 2 · Database

Option A: MariaDB (default)

```
sudo apt install -y mariadb-server
sudo systemctl enable --now mariadb
sudo mysql_secure_installation
sudo mysql -e "CREATE DATABASE datahub_catalog CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;"
sudo mysql -e "CREATE USER 'datahub'@'localhost' IDENTIFIED BY '<db-password>';"
sudo mysql -e "GRANT ALL PRIVILEGES ON datahub_catalog.* TO 'datahub'@'localhost';"
sudo mysql -e "FLUSH PRIVILEGES;"
```

> **Note:** The application creates all tables itself on first start. The MariaDB server also installs `mysqldump`, which the backup feature uses for an additional full SQL dump.

Option B: PostgreSQL

```
sudo apt install -y postgresql
sudo -u postgres psql -c "CREATE USER datahub WITH PASSWORD '<db-password>';"
sudo -u postgres psql -c "CREATE DATABASE datahub_catalog OWNER datahub;"
sudo -u postgres psql -d datahub_catalog -c "CREATE EXTENSION IF NOT EXISTS pg_trgm;"
```

In step 7 also set `DB_DRIVER=postgres` and `DB_PORT=5432`. The `pg_trgm` extension speeds up substring search; without it search still works, only slower.

Step 3 · Unpack the package and build the UI

```
mkdir -p ~/datahub-catalog
cd ~ && rm -rf datahub-catalog-release && mkdir datahub-catalog-release && cd \
    datahub-catalog-release
unzip -q ~/datahub-catalog-deploy.zip
rsync -a --delete --exclude .git --exclude node_modules --exclude dist --exclude venv --exclude \
    inspector/venv ~/datahub-catalog-release/datahub-catalog/ ~/datahub-catalog/
cd ~/datahub-catalog
grep '"version"' package.json
npm install
npm run build
```

`grep` must show `"version": "1.79.6"`. `npm run build` creates the `dist` directory with the UI. If you work with the Git repository, clone it to `~/datahub-catalog` instead and skip unzip and rsync.

Step 4 · Copy the UI to the web root

```
sudo mkdir -p /var/www/datahub-catalog
sudo cp -r dist/* /var/www/datahub-catalog/
```

Step 5 · Install the application server

```
sudo mkdir -p /opt/datahub-catalog
sudo cp server.js mcp.js okf.js catalog-diff.js csv-import.js csv-export.js permissions.js \
    uc-writeback.js package.json package-lock.json /opt/datahub-catalog/
sudo cp -r server src mcp /opt/datahub-catalog/
cd /opt/datahub-catalog && sudo npm install --omit=dev
cd ~/datahub-catalog
```

> **Note:** The server needs all of these files and the three directories `server`, `src` and `mcp`. If one is missing, start-up fails with “Cannot find module …”.

Step 6 · Install the source inspector (optional)

The inspector is only needed to read DuckDB files, data lake files (Parquet, CSV, JSON, Delta, Iceberg), SQL databases and JDBC sources. OData/SAP SuccessFactors, Kafka Schema Registry, Power BI, Informatica, dbt and ODCS run directly in the Node server. Without the inspector, skip this step and set `INSPECTOR_URL=` (empty) in step 7 – the affected source types then disappear from the UI.

```
sudo mkdir -p /opt/datahub-catalog/inspector /opt/datahub-catalog/jdbc-drivers
sudo cp inspector/app.py inspector/requirements.txt inspector/requirements.lock \
    inspector/requirements-optional.txt /opt/datahub-catalog/inspector/
sudo cp -r inspector/connectors /opt/datahub-catalog/inspector/
sudo python3 -m venv /opt/datahub-catalog/inspector/venv
sudo /opt/datahub-catalog/inspector/venv/bin/pip install -r \
    /opt/datahub-catalog/inspector/requirements.lock
sudo cp deploy/datahub-catalog-inspector.service /etc/systemd/system/
sudo systemctl daemon-reload
sudo systemctl enable --now datahub-catalog-inspector
curl -s http://127.0.0.1:3001/health
```

Expected: `{"ok":true}`. Install additional drivers into the same environment as needed:

Source	Installation (in the inspector venv)
MySQL/MariaDB, PostgreSQL, DuckDB, data lake files	included (DuckDB loads httpfs/azure/delta/iceberg on first use; needs internet access once)

Source	Installation (in the inspector venv)
Microsoft SQL Server / Azure SQL	`sudo /opt/datahub-catalog/inspector/venv/bin/pip install pymssql`
Oracle (thin mode, no Oracle client)	`… pip install oracledb`
Snowflake	`… pip install snowflake-sqlalchemy`
SAP HANA	`… pip install sqlalchemy-hana hdbcli`
IBM DB2 (x86_64 only)	`… pip install ibm-db-sa`
Generic JDBC (any database with a JDBC driver)	`sudo apt install -y default-jre-headless`, then `… pip install JPype1 jaydebeapi`; copy the driver .jar to `/opt/datahub-catalog/jdbc-drivers`

Note: ARM64 (e.g. Orange Pi): `ibm_db` does not build there. For DB2 use the JDBC route with IBM's pure Java driver (`com.ibm.db2:jcc` from Maven Central, saved as `db2jcc4.jar`; class `com.ibm.db2.jcc.DB2Driver`, URL `jdbc:db2://host:50000/DB`).

Step 7 · Configure and start the application server

```
openssl rand -base64 48
sudo cp deploy/datahub-catalog-proxy.service /etc/systemd/system/
sudo vim /etc/systemd/system/datahub-catalog-proxy.service
```

The first command generates a key for `SECRETS_KEY`. In the unit file set at least:

```
Environment=APP_URL=https://catalog.example.com
Environment=COOKIE_SECURE=0
Environment=SECRETS_KEY=<key-from-openssl>
Environment=DB_HOST=127.0.0.1
Environment=DB_PORT=3306
Environment=DB_USER=datahub
Environment=DB_PASSWORD=<db-password>
Environment=DB_NAME=datahub_catalog
```

Variable	Meaning
`APP_URL`	public address; used for links in invitations and e-mails and as the OAuth issuer for MCP clients
`COOKIE_SECURE`	set to 1 only after step 9 (HTTPS), otherwise sign-in over http fails
`SECRETS_KEY`	encrypts stored Databricks and Entra secrets (AES-256-GCM); set it once and keep it – without it stored secrets can no longer be read
`DB_DRIVER`	PostgreSQL only: `postgres` (then `DB_PORT=5432`)
`INSPECTOR_URL`	without the inspector set it explicitly to empty: `Environment=INSPECTOR_URL=`; if unset, http://127.0.0.1:3001 applies
`SMTP_HOST/PORT/USER/PASS/FROM`	optional: invitations, password links and notifications by e-mail; without SMTP the UI shows links to copy
`MCP_TOKEN`	optional: static bearer token for AI agents (`openssl rand -base64 32`); alternatively agent keys or OAuth (see step 12)
`ADMIN_EMAIL/ADMIN_PASSWORD`	optional: creates the first administrator at start-up while no users exist
`BACKUP_STORAGE`	`local` (default, `/var/lib/datahub-catalog/backups`), `s3`, `azure` or `databricks`

Further, rarely needed variables (rate limits, connector security, external scheduler, proxy sign-in) are documented as comments in the unit file. Then start:

```
sudo systemctl daemon-reload
sudo systemctl enable --now datahub-catalog-proxy
sudo journalctl -u datahub-catalog-proxy -n 20 --no-pager
```

Expected line: `Alassio Data Catalog v1.79.6 listening on http://127.0.0.1:3000 (mariadb: datahub_catalog, scheduler internal)`. “Access denied” or “ECONNREFUSED” means the database credentials or host do not match step 2.

Step 8 · Set up nginx

```
sudo cp deploy/nginx-datahub-catalog.conf /etc/nginx/sites-available/datahub-catalog
sudo vim /etc/nginx/sites-available/datahub-catalog
sudo ln -sf /etc/nginx/sites-available/datahub-catalog /etc/nginx/sites-enabled/datahub-catalog
sudo rm -f /etc/nginx/sites-enabled/default
sudo nginx -t
sudo systemctl reload nginx
```

In the file, replace `server_name _;` with your host name. The template already contains everything needed:

- Forwarding of `/auth`, `/api`, `/admin`, `/mcp`, `/oauth`, `/.well-known` and `/healthz` to the Node server.
- A separate block with a 512 MB upload limit and a 10-minute timeout for CSV import, exports, dump/restore and source inspection. It must stay above the general block.
- Security headers (Content-Security-Policy, X-Frame-Options, nosniff, Referrer and Permissions Policy) and exclusion from search engines.

Step 9 · HTTPS with Let's Encrypt

```
sudo apt install -y certbot python3-certbot-nginx
sudo certbot --nginx -d catalog.example.com
sudo sed -i 's/^Environment=COOKIE_SECURE=0/Environment=COOKIE_SECURE=1/' \
    /etc/systemd/system/datahub-catalog-proxy.service
sudo systemctl daemon-reload
sudo systemctl restart datahub-catalog-proxy
```

Once HTTPS works for every host name of the server, also enable the prepared `Strict-Transport-Security` line in the nginx file and reload nginx.

Step 10 · Firewall (if ufw is used)

```
sudo ufw allow OpenSSH
sudo ufw allow 'Nginx Full'
sudo ufw enable
```

Step 11 · First sign-in and initial setup

Open the address in a browser. While no users exist, “Create the first administrator” appears (name, e-mail, password with at least 8 characters). Afterwards you can set up two-factor authentication with a QR code or do it later via the key icon. Recommended first steps:

- **Invite users** (Admin › Users) and assign roles: Data Steward, Project Manager, Business Analyst, Data Ingestion, Development & Test / Sign-off, Guest.
- **Create the structure** or import it: data hub types (pre-set: Finance, HR, Sales and SOURCE), data hubs, databases – via Import › Unity Catalog, Import › CSV or Import › Load source.
- **Maintain responsibilities**: BPO per data hub, owner and steward per data domain (Governance Overview).
- **Check backups**: Admin › Backups & jobs – time, retention, then “Back up now” and “Verify” once.
- Optionally configure **Entra ID** (Microsoft sign-in) and the group → role mapping under Admin › Users.

Step 12 · Connect AI agents (optional, MCP)

The catalog is a Model Context Protocol server at `https://<host>/mcp`. There are three ways to authenticate:

- **Agent key** (recommended): Admin › Users › Invite Agent – name, read or write access; the key is shown once.
- **OAuth 2.1**: clients such as Claude sign in as the respective user (discovery, dynamic registration, PKCE); write access only for admins and data stewards.
- **Static token**: `MCP_TOKEN` in the unit file.

```
curl -s https://catalog.example.com/mcp -H "Authorization: Bearer <key>" -H \
    "Content-Type: application/json" -d '{"jsonrpc":"2.0","id":1,"method":"tools/list"}'
```

The response lists the tools (read, search, lineage, audit, catalog maintenance, glossary, stewardship, OKF export). Clients that only speak stdio connect via `npx mcp-remote https://catalog.example.com/mcp --header "Authorization: Bearer <key>"`.

Step 13 · Checklist

```
systemctl status datahub-catalog-proxy datahub-catalog-inspector --no-pager
curl -s http://127.0.0.1:3000/healthz
curl -s http://127.0.0.1:3001/health
sudo mysql datahub_catalog -e 'SHOW TABLES;'
```

- Both services are “active (running)”; `/healthz` returns `{"ok":true,"status":"ready","version":"1.79.6"}`.
- `SHOW TABLES` lists, among others, `users`, `sessions`, `audit`, `cat_hubs`, `cat_tables`, `cat_attributes`.
- In the browser: the dashboard after sign-in; the help (i icon) shows version 1.79.6 (Ctrl+F5 if an older one appears).
- A change to a database survives a reload and appears in the audit trail.

Updating to a new version

Every release ships as a complete package. Put the new `datahub-catalog-deploy.zip` into the home directory and run:

```
cd ~ && rm -rf datahub-catalog-release && mkdir datahub-catalog-release && cd \
    datahub-catalog-release
unzip -q ~/datahub-catalog-deploy.zip
rsync -a --delete --exclude .git --exclude node_modules --exclude dist --exclude venv --exclude \
    inspector/venv ~/datahub-catalog-release/datahub-catalog/ ~/datahub-catalog/
cd ~/datahub-catalog
npm install
npm run build
sudo cp -r dist/* /var/www/datahub-catalog/
sudo cp server.js mcp.js okf.js catalog-diff.js csv-import.js csv-export.js permissions.js \
    uc-writeback.js package.json package-lock.json /opt/datahub-catalog/
sudo cp -r server src mcp /opt/datahub-catalog/
sudo cp inspector/app.py inspector/requirements.txt inspector/requirements.lock \
    /opt/datahub-catalog/inspector/
sudo cp -r inspector/connectors /opt/datahub-catalog/inspector/
cd /opt/datahub-catalog && sudo npm install --omit=dev
sudo systemctl restart datahub-catalog-inspector
sudo systemctl restart datahub-catalog-proxy
sleep 5
curl -s http://127.0.0.1:3000/healthz
```

The server applies database changes itself at start-up. Without the inspector, leave out the two `inspector` lines and its restart. If Python dependencies change, also run `sudo /opt/datahub-catalog/inspector/venv/bin/pip install -r /opt/datahub-catalog/inspector/requirements.lock`.

> **Caution:** Do not run `npm audit fix --force` on the server – it installs incompatible major versions. Security fixes to dependencies come with the next release.

Backup and restore

- **Automatic**: Admin › Backups & jobs writes a gzip'd catalog dump every day and – when `mysqldump` or `pg_dump` is available – an SQL dump of the whole database. Time and retention are configurable.
- **Verify** runs a restore drill (structure, referential integrity, record counts).
- **Restore** replays a backup through the normal restore job; downloads hand you the files.
- For backups off the server set `BACKUP_STORAGE=s3` (S3_BUCKET, S3_REGION, S3_ACCESS_KEY_ID, S3_SECRET_ACCESS_KEY, optionally S3_ENDPOINT for MinIO) or `azure` (AZURE_CONTAINER_SAS_URL).

Troubleshooting

Symptom	Cause and fix
Browser shows an old version	package not re-extracted or browser cache: check the version in `package.json`, rebuild, Ctrl+F5
502 Bad Gateway	application server not running: `sudo journalctl -u datahub-catalog-proxy -n 50 --no-pager`
“Cannot find module …”	a file or directory is missing in /opt (step 5) or `npm install --omit=dev` was not run there
Sign-in fails immediately, no cookie	`COOKIE_SECURE=1` without HTTPS – set it to 1 only after step 9
403 “csrf” on changes	origin does not match the host: nginx must pass `proxy_set_header Host $host` (included in the template); then reload the page
MCP returns HTML instead of JSON	nginx block without `mcp` – use the template from `deploy/`
“inspector service is not reachable”	`systemctl status datahub-catalog-inspector`; the venv must be at `/opt/datahub-catalog/inspector/venv`
Uploads of large files abort	the nginx block for large transfers is missing or below the general block
Two-factor device lost	`sudo mysql datahub_catalog -e "UPDATE users SET totp_enabled=0, totp_secret=NULL WHERE email='user@example.com';"`